Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет прикладной информатики
Образовательная программа Мобильные и сетевые технологии
Направление подготовки 09.03.03 Мобильные и сетевые технологии

ОТЧЕТ по Лабораторной работе № 5

по дисциплине «Проектирование и реализация баз данных»

Обучающийся: Мельникова Настасья Филипповна К3241

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург,
2025

Цель работы

Приобрести практические навыки создания и использования процедур, функций и триггеров в PostgreSQL

Практическое задание:

По варианту индивидуальной БД (ЛР 2, часть IV) необходимо:

1. Создать три хранимые процедуры (CALL).

2. Разработать семь оригинальных триггеров (AFTER/BEFORE, ROW).

3. Проверить работу объектов в консоли psql, сделать скриншоты.

Схема базы данных

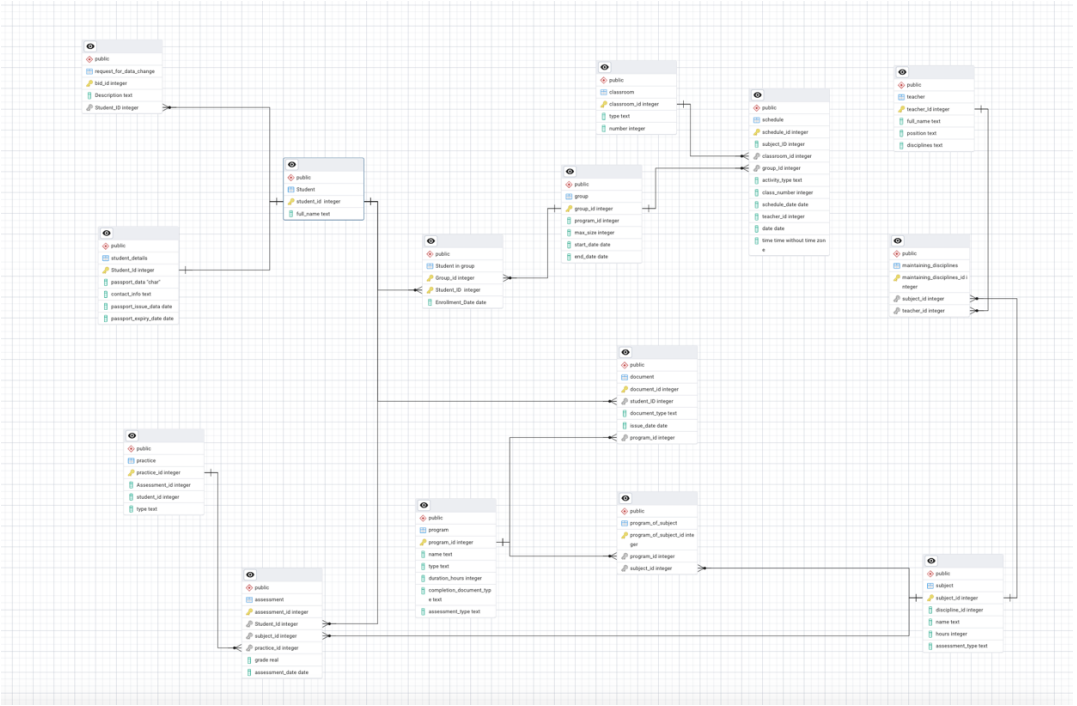


Рис.1: БД «Сессия»

Разработка хранимых процедур

1. Выполняет три связанные операции в одной команде — добавляет полную информацию о новом студенте в базу данных: Добавляет нового студента, Добавляет паспортные данные, Зачисляет студента в группу

```
postgres=# CREATE OR REPLACE PROCEDURE pr_add_student_full(
postgres(#     IN p_full_name TEXT,
postgres(#     IN p_passport_data CHAR(10),
postgres(#     IN p_contact_info TEXT,
postgres(#     IN p_issue_date DATE,
postgres(#     IN p_expiry_date DATE,
postgres(#     IN p_group_id INTEGER,
postgres(#     IN p_enroll_date DATE
postgres(# )
postgres-# LANGUAGE plpgsql
postgres-# AS $$
postgres$# DECLARE
postgres$#     new_id INTEGER;
postgres$# BEGIN
postgres$#     INSERT INTO "Student"(full_name)
postgres$#     VALUES (p_full_name)
postgres$#     RETURNING "student_id " INTO new_id;
postgres$#
postgres$#     INSERT INTO student_details(
postgres$#         "Student_Id", passport_data, contact_info,
postgres$#         passport_issue_data, passport_expiry_date
postgres$#     )
postgres$#     VALUES (
postgres$#         new_id, p_passport_data, p_contact_info,
postgres$#         p_issue_date, p_expiry_date
postgres$#     );
postgres$#
postgres$#     INSERT INTO "Student in group"(
postgres$#         "Group_id", "Student_ID ", "Enrollment_Date"
postgres$#     )
postgres$#     VALUES (
postgres$#         p_group_id, new_id, p_enroll_date
postgres$#     );
postgres$#
postgres$#     RAISE NOTICE 'Студент успешно добавлен с id %', new_id;
postgres$# END;
postgres$# $$;
CREATE PROCEDURE
```

```
CREATE OR REPLACE PROCEDURE pr_add_student_full(
    IN p_full_name TEXT,
    IN p_passport_data CHAR(10),
    IN p_contact_info TEXT,
    IN p_issue_date DATE,
    IN p_expiry_date DATE,
    IN p_group_id INTEGER,
    IN p_enroll_date DATE
)
LANGUAGE plpgsql
AS $$
DECLARE
    new_id INTEGER;
BEGIN
    INSERT INTO "Student"(full_name)
    VALUES (p_full_name)
    RETURNING "student_id " INTO new_id;

    INSERT INTO student_details(
        "Student_Id", passport_data, contact_info,
        passport_issue_data, passport_expiry_date
    )
    VALUES (
        new_id, p_passport_data, p_contact_info,
        p_issue_date, p_expiry_date
    );

    INSERT INTO "Student in group"(
        "Group_id", "Student_ID ", "Enrollment_Date"
    )
    VALUES (
        p_group_id, new_id, p_enroll_date
    );

    RAISE NOTICE 'Студент успешно добавлен с id %', new_id;
END;
$$;
```

```
postgres=# CALL pr_add_student_full(
postgres(#     'Даркмевич Мария Сергеевна',
postgres(#     '1112223334',
postgres(#     'm.dark@example.com',
postgres(#     '2022-01-01',
postgres(#     '2032-01-01',
postgres(#     102,
postgres(#     '2023-09-01'
postgres(# );
NOTICE:  Студент успешно добавлен с id 17
```

```
postgres=# SELECT * FROM "Student" ORDER BY "student_id " DESC LIMIT 1;
 student_id  |         full_name
-------------+---------------------------
          17 | Даркмевич Мария Сергеевна
(1 row)

postgres=# SELECT * FROM student_details ORDER BY "Student_Id" DESC LIMIT 1;
 Student_Id | passport_data |    contact_info    | passport_issue_data | passport_expiry_date
------------+---------------+--------------------+---------------------+----------------------
         17 | 1112223334    | m.dark@example.com | 2022-01-01          | 2032-01-01
(1 row)

postgres=# SELECT * FROM "Student in group" ORDER BY "Student_ID " DESC LIMIT 1;
 Group_id | Student_ID  | Enrollment_Date
----------+-------------+-----------------
      102 |          17 | 2023-09-01
(1 row)
```

2. Проверяет, есть ли уже студент с таким ФИО и паспортом

```
postgres=# CREATE OR REPLACE PROCEDURE pr_register_student_if_not_exists(
postgres(#     IN p_full_name TEXT,
postgres(#     IN p_passport_data CHAR(10),
postgres(#     IN p_contact_info TEXT,
postgres(#     IN p_issue_date DATE,
postgres(#     IN p_expiry_date DATE,
postgres(#     IN p_group_id INTEGER,
postgres(#     IN p_enroll_date DATE
postgres(# )
postgres-# LANGUAGE plpgsql
postgres-# AS $$
postgres$# DECLARE
postgres$#     existing_id INTEGER;
postgres$#     new_id INTEGER;
postgres$# BEGIN
postgres$#     SELECT s."student_id "
postgres$#     INTO existing_id
postgres$#     FROM "Student" s
postgres$#     JOIN student_details sd ON s."student_id " = sd."Student_Id"
postgres$#     WHERE s.full_name = p_full_name AND sd.passport_data = p_passport_data;
postgres$#
postgres$#     IF existing_id IS NOT NULL THEN
postgres$#         RAISE NOTICE 'Студент с таким ФИО и паспортом уже существует. id = %', existing_id;
postgres$#         RETURN;
postgres$#     END IF;
postgres$#
postgres$#     INSERT INTO "Student"(full_name)
postgres$#     VALUES (p_full_name)
postgres$#     RETURNING "student_id " INTO new_id;
postgres$#
postgres$#     INSERT INTO student_details(
postgres$#         "Student_Id", passport_data, contact_info,
postgres$#         passport_issue_data, passport_expiry_date
postgres$#     )
postgres$#     VALUES (
postgres$#         new_id, p_passport_data, p_contact_info,
postgres$#         p_issue_date, p_expiry_date
postgres$#     );
postgres$#
postgres$#     -- Запись в группу
postgres$#     INSERT INTO "Student in group"(
postgres$#         "Group_id", "Student_ID ", "Enrollment_Date"
postgres$#     )
postgres$#     VALUES (
postgres$#         p_group_id, new_id, p_enroll_date
postgres$#     );
postgres$#
postgres$#     RAISE NOTICE 'Добавлен новый студент с id %', new_id;
postgres$# END;
postgres$# $$;
CREATE PROCEDURE
```

```
CREATE OR REPLACE PROCEDURE pr_register_student_if_not_exists(
    IN p_full_name TEXT,
    IN p_passport_data CHAR(10),
    IN p_contact_info TEXT,
```

```
    IN p_issue_date DATE,
    IN p_expiry_date DATE,
    IN p_group_id INTEGER,
    IN p_enroll_date DATE
)
LANGUAGE plpgsql
AS $$
DECLARE
    existing_id INTEGER;
    new_id INTEGER;
BEGIN
    SELECT s."student_id "
    INTO existing_id
    FROM "Student" s
    JOIN student_details sd ON s."student_id " = sd."Student_Id"
    WHERE s.full_name = p_full_name AND sd.passport_data =
p_passport_data;

    IF existing_id IS NOT NULL THEN
        RAISE NOTICE 'Студент с таким ФИО и паспортом уже существует.
id = %', existing_id;
        RETURN;
    END IF;

    INSERT INTO "Student"(full_name)
    VALUES (p_full_name)
    RETURNING "student_id " INTO new_id;

    INSERT INTO student_details(
        "Student_Id", passport_data, contact_info,
        passport_issue_data, passport_expiry_date
    )
    VALUES (
        new_id, p_passport_data, p_contact_info,
        p_issue_date, p_expiry_date
    );

    -- Запись в группу
    INSERT INTO "Student in group"(
        "Group_id", "Student_ID ", "Enrollment_Date"
    )
    VALUES (
        p_group_id, new_id, p_enroll_date
    );

    RAISE NOTICE 'Добавлен новый студент с id %', new_id;
END;
$$;
```

```
postgres=# CALL pr_register_student_if_not_exists(
postgres(#     'Сидорова Анна Викторовна',
postgres(#     '6',
postgres(#     'testov@example.com',
postgres(#     '2022-01-01',
postgres(#     '2032-01-01',
postgres(#     101,
postgres(#     '2023-09-01'
postgres(# );
NOTICE:  Студент с таким ФИО и паспортом уже существует. id = 3
CALL
```

2. Назначает преподавателя на пару в расписании, **проверяет**, нет ли уже такой пары (с тем же предметом, группой, временем и датой), **если пары нет**, добавляет её в таблицу schedule

```
postgres=# CREATE OR REPLACE PROCEDURE pr_assign_teacher_to_schedule_subject(
postgres(#     IN p_teacher_id INTEGER,
postgres(#     IN p_subject_id INTEGER,
postgres(#     IN p_group_id INTEGER,
postgres(#     IN p_classroom_id INTEGER,
postgres(#     IN p_class_number INTEGER,
postgres(#     IN p_activity_type TEXT,
postgres(#     IN p_schedule_date DATE,
postgres(#     IN p_time TIME
postgres(# )
postgres-# LANGUAGE plpgsql
postgres-# AS $$
postgres$# BEGIN
postgres$#     IF EXISTS (
postgres$#         SELECT 1 FROM schedule
postgres$#         WHERE "subject_ID" = p_subject_id
postgres$#           AND "group_Id" = p_group_id
postgres$#           AND schedule_date = p_schedule_date
postgres$#           AND time = p_time
postgres$#     ) THEN
postgres$#         RAISE NOTICE 'Пара уже существует в это время.';
postgres$#         RETURN;
postgres$#     END IF;
postgres$#
postgres$#     INSERT INTO schedule(
postgres$#         "subject_ID", classroom_id, "group_Id",
postgres$#         class_number, activity_type,
postgres$#         schedule_date, date, time, teacher_id
postgres$#     )
postgres$#     VALUES (
postgres$#         p_subject_id, p_classroom_id, p_group_id,
postgres$#         p_class_number, p_activity_type,
postgres$#         p_schedule_date, p_schedule_date, p_time, p_teacher_id
postgres$#     );
postgres$#
postgres$#     RAISE NOTICE 'Пара успешно добавлена.';
postgres$# END;
postgres$# $$;
CREATE PROCEDURE
```

```
CREATE OR REPLACE PROCEDURE pr_assign_teacher_to_schedule_subject(
    IN p_teacher_id INTEGER,
    IN p_subject_id INTEGER,
    IN p_group_id INTEGER,
    IN p_classroom_id INTEGER,
    IN p_class_number INTEGER,
    IN p_activity_type TEXT,
    IN p_schedule_date DATE,
    IN p_time TIME
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM schedule
        WHERE "subject_ID" = p_subject_id
          AND "group_Id" = p_group_id
          AND schedule_date = p_schedule_date
          AND time = p_time
```

```
    ) THEN
        RAISE NOTICE 'Пара уже существует в это время.';
        RETURN;
    END IF;

    INSERT INTO schedule(
        "subject_ID", classroom_id, "group_Id",
        class_number, activity_type,
        schedule_date, date, time, teacher_id
    )
    VALUES (
        p_subject_id, p_classroom_id, p_group_id,
        p_class_number, p_activity_type,
        p_schedule_date, p_schedule_date, p_time, p_teacher_id
    );

    RAISE NOTICE 'Пара успешно добавлена.';
END;
$$;
```

```
postgres=# CALL pr_assign_teacher_to_schedule_subject(
    2,
    102,
    201,
    2,
    3,
    'лекция',
    '2024-09-10',
    '10:40:00'
);
NOTICE:  Пара успешно добавлена.
CALL
```

```
postgres=# SELECT * FROM schedule
postgres-# WHERE schedule_date = '2024-09-10' AND time = '10:40:00';
 schedule_id | subject_ID | classroom_id | group_Id | activity_type | class_number | schedule_date | teacher_id |    date    |   time
-------------+------------+--------------+----------+---------------+--------------+---------------+------------+------------+----------
          10 |        102 |            2 |      201 | лекция        |            3 | 2024-09-10    |          2 | 2024-09-10 | 10:40:00
(1 row)
```

Разработка триггеров

Триггер 1: проверка уникальности названия предмета

```
postgres=# CREATE OR REPLACE FUNCTION trg_prevent_duplicate_subject_names()
postgres-# RETURNS TRIGGER AS $$
postgres$# BEGIN
postgres$#     IF EXISTS (
postgres$#         SELECT 1 FROM subject
postgres$#         WHERE name = NEW.name
postgres$#           AND subject_id <> NEW.subject_id
postgres$#     ) THEN
postgres$#         RAISE EXCEPTION 'Предмет с таким названием уже существует';
postgres$#     END IF;
postgres$#     RETURN NEW;
postgres$# END;
postgres$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=#
postgres=# CREATE TRIGGER trg_unique_subject_name
postgres-# BEFORE INSERT OR UPDATE ON subject
postgres-# FOR EACH ROW
postgres-# EXECUTE FUNCTION trg_prevent_duplicate_subject_names();
CREATE TRIGGER
```

CREATE OR REPLACE FUNCTION trg_prevent_duplicate_subject_names()

RETURNS TRIGGER AS $$

BEGIN

IF EXISTS (

SELECT 1 FROM subject

WHERE name = NEW.name

AND subject_id <> NEW.subject_id

) THEN

RAISE EXCEPTION 'Предмет с таким названием уже существует';

END IF;

RETURN NEW;

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_unique_subject_name

BEFORE INSERT OR UPDATE ON subject

FOR EACH ROW

EXECUTE FUNCTION trg_prevent_duplicate_subject_names();

```
postgres=# INSERT INTO subject(subject_id, name)
postgres-# VALUES (999, 'Математика');
ERROR:  Предмет с таким названием уже существует
```

Триггер 2: Контроль оценок: только от 2 до 5

```
postgres=# CREATE OR REPLACE FUNCTION trg_validate_grade_range()
postgres-# RETURNS TRIGGER AS $$
postgres$# BEGIN
postgres$#     IF NEW.grade < 2 OR NEW.grade > 5 THEN
postgres$#         RAISE EXCEPTION 'Оценка должна быть от 2 до 5';
postgres$#     END IF;
postgres$#     RETURN NEW;
postgres$# END;
postgres$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=#
postgres=# CREATE TRIGGER trg_check_grade_range
postgres-# BEFORE INSERT OR UPDATE ON assessment
postgres-# FOR EACH ROW
postgres-# EXECUTE FUNCTION trg_validate_grade_range();
CREATE TRIGGER
```

CREATE OR REPLACE FUNCTION trg_validate_grade_range()

RETURNS TRIGGER AS $$

BEGIN

IF NEW.grade < 2 OR NEW.grade > 5 THEN

RAISE EXCEPTION 'Оценка должна быть от 2 до 5';

END IF;

RETURN NEW;

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_grade_range

BEFORE INSERT OR UPDATE ON assessment

FOR EACH ROW

EXECUTE FUNCTION trg_validate_grade_range();

```
postgres=# INSERT INTO assessment(assessment_id,"Student_Id", subject_id, grade)
VALUES (1001, 1, 101, 1);
ERROR:  Оценка должна быть от 2 до 5
```

Триггер 3: Запрет двойной регистрации студента в одной группе

```
postgres=# CREATE OR REPLACE FUNCTION trg_prevent_duplicate_group_registration()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM "Student in group"
        WHERE "Student_ID " = NEW."Student_ID "
          AND "Group_id" = NEW."Group_id"
    ) THEN
        RAISE EXCEPTION 'Студент уже добавлен в эту группу';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
```

```
postgres=# CREATE TRIGGER trg_prevent_duplicate_group
postgres-# BEFORE INSERT ON "Student in group"
postgres-# FOR EACH ROW
postgres-# EXECUTE FUNCTION trg_prevent_duplicate_group_registration();
CREATE TRIGGER
```

CREATE OR REPLACE FUNCTION trg_prevent_duplicate_group_registration()
RETURNS TRIGGER AS $$
BEGIN
   IF EXISTS (
      SELECT 1 FROM "Student in group"
      WHERE "Student_ID " = NEW."Student_ID "
        AND "Group_id" = NEW."Group_id"
) THEN
      RAISE EXCEPTION 'Студент уже добавлен в эту группу';
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_duplicate_group

BEFORE INSERT ON "Student in group"
FOR EACH ROW
EXECUTE FUNCTION trg_prevent_duplicate_group_registration();

```
postgres=# INSERT INTO "Student in group"("Student_ID ", "Group_id")
VALUES (17, 102);
ERROR:  Студент уже добавлен в эту группу
```

Триггер 4: Ограничивает количество студентов в группе до 30

```
postgres=# CREATE OR REPLACE FUNCTION trg_limit_students_per_group()
postgres-# RETURNS TRIGGER AS $$
postgres$# DECLARE
postgres$#     count_students INT;
postgres$# BEGIN
postgres$#     SELECT COUNT(*) INTO count_students
postgres$#     FROM "Student in group"
postgres$#     WHERE "Group_id" = NEW."Group_id";
postgres$#
postgres$#     IF count_students >= 30 THEN
postgres$#         RAISE EXCEPTION 'В группе уже 30 студентов';
postgres$#     END IF;
postgres$#
postgres$#     RETURN NEW;
postgres$# END;
postgres$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=#
postgres=# CREATE TRIGGER trg_max_students_in_group
postgres-# BEFORE INSERT ON "Student in group"
postgres-# FOR EACH ROW
postgres-# EXECUTE FUNCTION trg_limit_students_per_group();
CREATE TRIGGER
```

```
CREATE OR REPLACE FUNCTION trg_limit_students_per_group()
RETURNS TRIGGER AS $$
DECLARE
    count_students INT;
BEGIN
    SELECT COUNT(*) INTO count_students
    FROM "Student in group"
    WHERE "Group_id" = NEW."Group_id";

    IF count_students >= 30 THEN
        RAISE EXCEPTION 'В группе уже 30 студентов';
    END IF;
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_max_students_in_group
BEFORE INSERT ON "Student in group"
FOR EACH ROW
EXECUTE FUNCTION trg_limit_students_per_group();
```

```
postgres=# SELECT "Group_id", COUNT(*) AS total
FROM "Student in group"
GROUP BY "Group_id"
ORDER BY total DESC;
 Group_id | total
----------+-------
      102 |     4
      101 |     4
      103 |     3
      203 |     1
      201 |     1
      202 |     1
(6 rows)
```

```
postgres=# INSERT INTO "Student in group"("Student_ID ", "Group_id", "Enrollment_Date")
VALUES (19, 102, '2024-09-01');
INSERT 0 1
```

Триггер 5: Запрещает менять группу у уже зачисленного студента

```
postgres=# CREATE OR REPLACE FUNCTION trg_block_group_change()
postgres-# RETURNS TRIGGER AS $$
postgres$# BEGIN
postgres$#     IF NEW."Group_id" <> OLD."Group_id" THEN
postgres$#         RAISE EXCEPTION 'Нельзя изменить группу после зачисления';
postgres$#     END IF;
postgres$#     RETURN NEW;
postgres$# END;
postgres$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=#
postgres=# CREATE TRIGGER trg_no_group_change
postgres-# BEFORE UPDATE ON "Student in group"
postgres-# FOR EACH ROW
postgres-# EXECUTE FUNCTION trg_block_group_change();
CREATE TRIGGER
```

```
CREATE OR REPLACE FUNCTION trg_block_group_change()
RETURNS TRIGGER AS $$
BEGIN
```

```
    IF NEW."Group_id" <> OLD."Group_id" THEN
        RAISE EXCEPTION 'Нельзя изменить группу после зачисления';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_no_group_change
BEFORE UPDATE ON "Student in group"
FOR EACH ROW
EXECUTE FUNCTION trg_block_group_change();
```

```
postgres=# SELECT * FROM "Student in group" WHERE "Student_ID " = 15;
 Group_id | Student_ID  | Enrollment_Date
----------+-------------+-----------------
      103 |          15 | 2023-09-01
(1 row)

postgres=# UPDATE "Student in group"
postgres-# SET "Group_id" = 102
postgres-# WHERE "Student_ID " = 15;
ERROR:  Нельзя изменить группу после зачисления
```

Триггер 6: запрет пересечения занятий у одной группы

```
postgres=# CREATE OR REPLACE FUNCTION trg_prevent_schedule_overlap()
postgres-# RETURNS TRIGGER AS $$
postgres$# BEGIN
postgres$#     IF EXISTS (
postgres$#         SELECT 1
postgres$#         FROM schedule
postgres$#         WHERE "group_Id" = NEW."group_Id"
postgres$#           AND schedule_date = NEW.schedule_date
postgres$#           AND time = NEW.time
postgres$#     ) THEN
postgres$#         RAISE EXCEPTION 'У группы уже есть занятие в это время';
postgres$#     END IF;
postgres$#     RETURN NEW;
postgres$# END;
postgres$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=#
postgres=# CREATE TRIGGER trg_check_schedule_overlap
postgres-# BEFORE INSERT ON schedule
postgres-# FOR EACH ROW
postgres-# EXECUTE FUNCTION trg_prevent_schedule_overlap();
CREATE TRIGGER
```

```
CREATE OR REPLACE FUNCTION trg_prevent_schedule_overlap()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
```

```
        SELECT 1
        FROM schedule
        WHERE "group_Id" = NEW."group_Id"
          AND schedule_date = NEW.schedule_date
          AND time = NEW.time
    ) THEN
        RAISE EXCEPTION 'У группы уже есть занятие в это время';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_schedule_overlap
BEFORE INSERT ON schedule
FOR EACH ROW
EXECUTE FUNCTION trg_prevent_schedule_overlap();
```

```
postgres=# SELECT * FROM schedule WHERE "group_Id" = 102;
 schedule_id | subject_ID | classroom_id | group_Id | activity_type | class_number | schedule_date | teacher_id |    date    |   time
-------------+------------+--------------+----------+---------------+--------------+---------------+------------+------------+----------
           6 |        104 |            3 |      102 | практика      |            1 | 2024-09-04    |          7 | 2024-09-04 | 09:00:00
           7 |        109 |            3 |      102 | практика      |            2 | 2024-09-06    |          6 | 2024-09-06 | 12:20:00
(2 rows)
```

```
postgres=# INSERT INTO schedule("subject_ID", classroom_id, "group_Id", class_number, activity_type, schedule_date, date, time, teacher_id)
VALUES (101, 1, 102, 3, 'лекция', '2024-09-06', '2024-09-06', '12:20:00', 5);
ERROR:  У группы уже есть занятие в это время
```

Триггер 7: защита от пересечения занятий преподавателя

```
postgres=# CREATE OR REPLACE FUNCTION trg_prevent_teacher_overlap()
postgres-# RETURNS TRIGGER AS $$
postgres$# BEGIN
postgres$#     IF EXISTS (
postgres$#         SELECT 1
postgres$#         FROM schedule
postgres$#         WHERE teacher_id = NEW.teacher_id
postgres$#           AND schedule_date = NEW.schedule_date
postgres$#           AND time = NEW.time
postgres$#     ) THEN
postgres$#         RAISE EXCEPTION 'У преподавателя уже есть занятие в это время';
postgres$#     END IF;
postgres$#     RETURN NEW;
postgres$# END;
postgres$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
postgres=#
postgres=# CREATE TRIGGER trg_check_teacher_overlap
postgres-# BEFORE INSERT ON schedule
postgres-# FOR EACH ROW
postgres-# EXECUTE FUNCTION trg_prevent_teacher_overlap();
CREATE TRIGGER
```

```
CREATE OR REPLACE FUNCTION trg_prevent_teacher_overlap()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM schedule
        WHERE teacher_id = NEW.teacher_id
          AND schedule_date = NEW.schedule_date
```

```
            AND time = NEW.time
    ) THEN
        RAISE EXCEPTION 'У преподавателя уже есть занятие в это
время';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_teacher_overlap
BEFORE INSERT ON schedule
FOR EACH ROW
EXECUTE FUNCTION trg_prevent_teacher_overlap();
```

```
postgres=# SELECT * FROM schedule WHERE teacher_id = 5;
 schedule_id | subject_ID | classroom_id | group_Id | activity_type | class_number | schedule_date | teacher_id |    date    |   time
-------------+------------+--------------+----------+---------------+--------------+---------------+------------+------------+----------
           5 |        108 |            2 |      101 | практика      |            2 | 2024-09-05    |          5 | 2024-09-05 | 10:40:00
(1 row)

postgres=# INSERT INTO schedule("subject_ID", classroom_id, "group_Id", class_number, activity_type, schedule_date, date, time, teacher_id)
postgres-# VALUES (103, 1, 4, 1, 'семинар', '2024-09-05', '2024-09-05', '10:40:00', 5);
ERROR:  У преподавателя уже есть занятие в это время
```

Выводы

В ходе выполнения работы:

• Разработала три процедуры

• Создала семь триггеров.

• Проверила их работоспособность в psql, результаты зафиксированы скриншотами.

Использование процедур и триггеров позволяет концентрировать логику на стороне СУБД, упрощая клиентские приложения и обеспечивая целостность данных.